
Task 4: Data Classification

Team Vectorized Engines

Simon Grundner (k12136610)

Mihael Zupanic (k12416041)

1 Dataset Preparation

1.1 Label Aggregation

We aggregate the annotation array [T, C, A] into a binary label matrix [T, C] using the `get_features_and_targets` function from the tutorial. The function first binarizes the overlap fractions with `(annotations > 0).astype(int)`, so any nonzero overlap counts as a yes vote from that annotator.

The votes are then summed along the annotator axis and thresholded with `votes > (A // 2)`, where A is the number of annotators in the file. This requires a strict majority — for A=2 both annotators must agree, for A=1 that single vote decides, and for higher A more than half must agree. We use the "multilabel" mode because multiple sound events can overlap in the same segment.

Counting any overlap as a yes vote is forgiving on segment boundaries, which matters since sound events rarely line up perfectly with the 1 second windows. The majority vote filters out individual annotator mistakes, like the missing keyboard_tying we found in 000278.wav in the previous task.

We are biased towards false negatives — on two-annotator files a single disagreement drops the label, which especially hurts classes with low Jaccard agreement like `door_open_close` (52.3%) and `wardrobe_drawer_open_close` (51.9%) (as seen in Task 3 of the project). Around 17% of files only have one annotator, so their labels are never checked. Binarizing also throws away the overlap fraction — 5% overlap and 95% overlap end up treated the same.

An alternative would be taking the union of annotators, but that would just swap our false negatives for false positives. One could also weight the confidence in the annotator, for example if one of the annotators is also the author, we could choose to trust the author upon disagreement.

1.2 Data Split

(i) Verbatim to the description of data leakage in the tutorial: “*Data leakage occurs when a model has access to information during training that would not be available at prediction time, leading to overly optimistic performance.*” Consecutive frames have similar feature characteristics (high correlation), if these consecutive samples land in the validation and training data, we will get a misleadingly high performance for the training of the model, although it performs poorly on new or unseen data.

Originally the data is split randomly using the `train_test_split()` method, for which data leakage is likely to occur. This is then improved, by selecting a whole adjacent section of the feature data array. This mitigates the information leakage effect, but still a large portion of our data is used for validation and not training.

To get a higher training effect from the data which is not used for testing, *k*-fold cross validation is used. This method sections the data which is not used for testing into *k* equal sets, and selects one as the validation set and the others as training sets. This is done *k* times while each iteration uses another validation set. Finally, the performance is averaged from iterations.

(ii) Equal class distribution is crucial, because a training set, which largely consists of a class which is not present in the validation set at all, would bring down performance. To ensure the classes are equally distributed across the *k* different data splits, a method called *stratified k*-fold cross validation is used [?]. A comparison of how the data is split can be seen in Figure 1. The method is later used implicitly by the builtin sklearn evaluation methods. The final split of the data is 90% training + cross validation data and 10% test data.

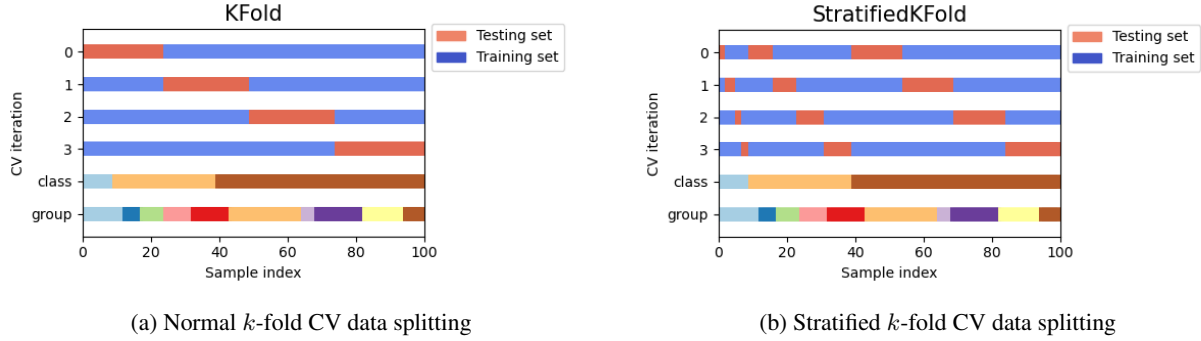


Figure 1: Comparison on equal data sectioning versus sectioning based on class frequency (stratified k -fold). Taken from the sklearn documentation.

1.3 Preprocessing

Loudness or energy can vary between audio files, so normalizing these features might help. The simplest way to standardize the features, is by removing the mean and scaling them to unit variance. This can be achieved by applying the *StandardScaler* from the sklearn. It is important that the same pre processing has to be applied when using the classifier in action.

2 Evaluation

2.1 Evaluation Criterion

In this case, the classification scenario is a multi-label classification, since there could be multiple annotations per audio segment, if two sounds events overlap. From the previous task (Task 3) one could see, that the classes are strongly imbalanced. To evaluate the performance of a classifier, a list of metrics shown in Table 1 can be taken into account.

Table 1: Reasoning on the chosen evaluation criterion

Metric	Category	Details
Accuracy	Description	% of correct predictions.
	When it's useful	Simple, but <i>misleading with imbalanced data</i> .
	Viable metric?	No , because the classes are strongly imbalanced. Yes for <i>balanced</i> accuracy.
Precision	Description	How many predicted positives were correct.
	When it's useful	Important when <i>false positives</i> are costly (e.g., spam filters).
	Viable metric?	No , a high precision model prefers to predict more negatives overall and only classifies a label as true if the event is very likely (high threshold). Infrequent classes will be predicted even more rarely.
Recall	Description	How many actual positives were caught.
	When it's useful	Critical when <i>false negatives</i> are costly (e.g., medical diagnosis).
	Viable metric?	No , prefers to predict more positives overall, implying more false positives (low threshold). Especially for multi-label classification it would be very likely for multiple false positives at once.
F1 Score	Description	Harmonic mean of precision and recall.
	When it's useful	Good balance when classes are imbalanced.
	Viable metric?	Yes , a balanced threshold is preferable here. On one hand we do not want the audio event detection to classify <i>everything</i> as a label, but neither no events at all.
ROC AUC	Description	Measures ranking ability over all thresholds.
	When it's useful	Good for probabilistic models; not sensitive to threshold.
	Viable metric?	No , since the data is strongly imbalanced.
PR AUC	Description	Average Precision. Area under Precision–Recall curve.

Table 1: Reasoning on the chosen evaluation criterion (continued)

Metric	Category	Details
PR AUC	When it's useful Viable metric?	Better than ROC AUC for imbalanced data, especially when pos. events are rare. Yes , as the model consists largely of negative events. Seen later in section 2.2.2.

2.2 Baseline Performance

2.2.1 Why majority class classifier is not ideal

A simple baseline classifier could be defined to predict the majority class. By extending the *BaselineClassifier* for Multiple outputs, we realize that the majority class over all audio event is actually, that no class is predicted at all. The scores of this classifier are shown in Table 2. For the baseline metrics, the whole dataset has been taken into account.

Table 2: Scores for the majority class baseline classifier

Macro Avg. Bal. Accuracy	0.5
Macro F1 Score	0 (or undefined, because there are no true positives, only true negatives.)
PR AUC	0.0680

2.2.2 Empirical Class Frequency Classifier

The task description suggests taking class frequencies into account and make predictions based on this. For this, the baseline classifier has been extended. The classifier does not just predict the majority class, but takes the mean of how often each class occurs. Upon prediction, a class has a probability of occurring in the binary output vector based on frequency in the training set. Figure 2 shows the probabilities for each class to be predicted from a sample in the validation set. The red line indicates where the original majority class baseline classifier would have sampled for a prediction; so in this case all zeros. The scores of this classifier are shown in Table 3.

Table 3: Scores for the class frequency baseline classifier

Macro Avg. Bal. Accuracy	0.5004
Macro F1 Score	0.0653
PR AUC	0.0668

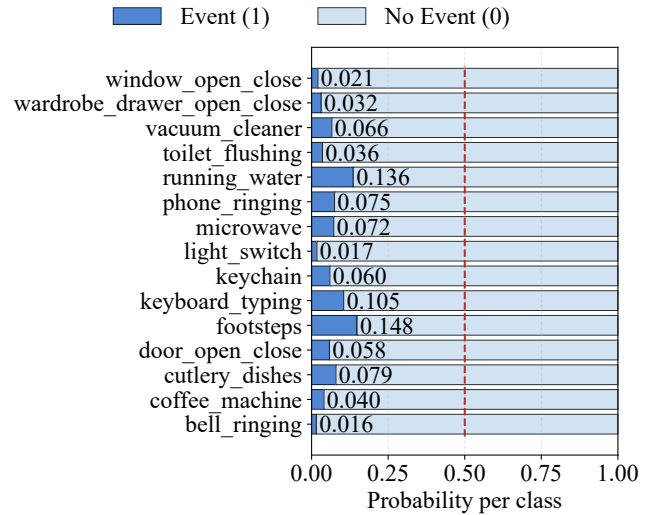


Figure 2: Probabilities for each class to be predicted, based on the class-frequency in the training data

It can be seen, that the F1 score has slightly increased, meaning the baseline classifier now predicts at least a few true positives. The equivalent builtin baseline classifier in sklearn is `DummyClassifier()` with the *stratified* strategy, which yields the same results as shown above. Let these results be the baseline performance to compare other classifiers with.

3 Experiments

The performance criterion to optimize the hyper parameters is chosen to be the *F1 score*. For the Experiments, the *Random Forest* classifier and a *Light Gradient Boosting Machine* were selected. To evaluate which parameters perform best, a grid search was used to iterate over the combination of hyper parameters of each classifier[?]. The data is split into 90% training data and 10% test data. The grid search performs a stratified¹ k-fold cross validation on the training data, therefor not manual split for the validation data was necessary. *k* was chosen to be 5.

¹*Stratified* meaning the cross validation is adapted to the class frequency. Better for imbalanced data as mentioned in Data Split

3.1 Random Forest Classifier

This classifier was chosen, because it was suggested as a task in the classifier tutorial. It was reasonably fast to train, even on the whole dataset, which simplified the procedure. The most relevant hyper parameters of the random forest classifier are defined as follows:

- **n_estimators**: Number of trees in the forest.
- **max_depth**: Maximum depth of a tree.
- **min_samples_split**: The minimum number of samples required to split an internal node

To systematically find the optimal hyperparameter vector, 20% of the dataset were used for the grid search. How the performance changes in the hyperspace can be seen in Figure 3. The maximum training F1 score is 0.258, for the hyperparameter vector: **n_estimators** = 5, **min_samples_split** = 5, **max_depth** = 20.

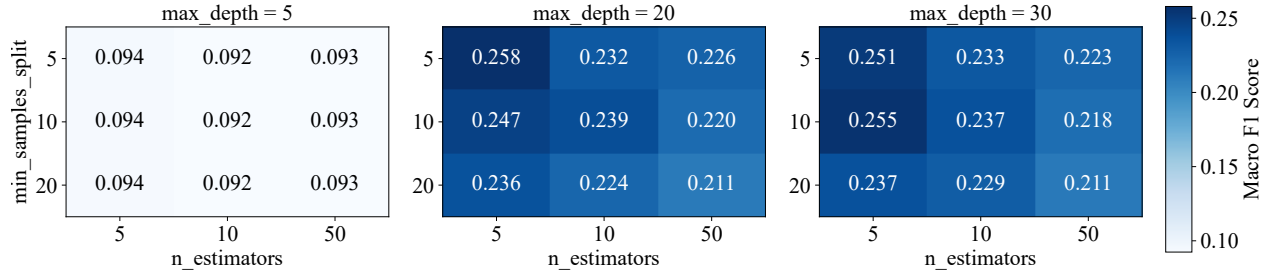


Figure 3: Hyperparameter Space of the random forest classifier. The darkest hyper parameter vector is chosen as the best model

Following these trends, a higher max_depth would indicate an even better performance. However, the training would be even more computationally expensive, so these parameters were accepted. It is interesting, that a lower number of total estimators perform better, which could be reasoned with overfitting.

The determined parameters were then used to train the random forest classifier with the whole dataset. The prediction score on the unbiased test set is shown in Table 4. A significant improvement can be seen in relation to the baseline performance.

Table 4: Scores for the random forest classifier predicting the unbiased test set

	Model	Baseline	Improvement Factor
Macro Avg. Bal. Accuracy	0.6334	0.5004	1.27
Macro F1 Score	0.3735	0.0653	5.72
PR AUC	0.2589	0.0668	3.88

3.2 Gradient Boosting Machine

Initially, we tried out Support Vector Machines, however a Non-Linear SVM took to long to train and the linear SVM performed very poorly, which is why we tried the Gradient Boosting Machine (GBM)[?] in this section. The most relevant hyper parameters of the GBM classifier are defined as follows:

- **n_estimators**: Number of boosted trees to fit.
- **max_depth**: Maximum tree depth for base learners.
- **learning_rate**: The minimum number of samples required to split an internal node

To systematically find the optimal hyperparameter vector, 20% of the dataset were used for the grid search. How the performance changes in the hyperspace can be seen in Figure 3. The maximum training F1 score is 0.526, for the hyperparameter vector: **n_estimators** = 100, **learning_rate** = 0.1, **max_depth** = 30.

As expected, a higher number of estimators as well as deeper trees perform better.

The determined parameters were then used to train the GBM classifier with the whole dataset. The prediction score on the unbiased test set is shown in Table 5. Again, a significant improvement can be seen in relation to the baseline performance, and a little improvement to the random forest classifier.

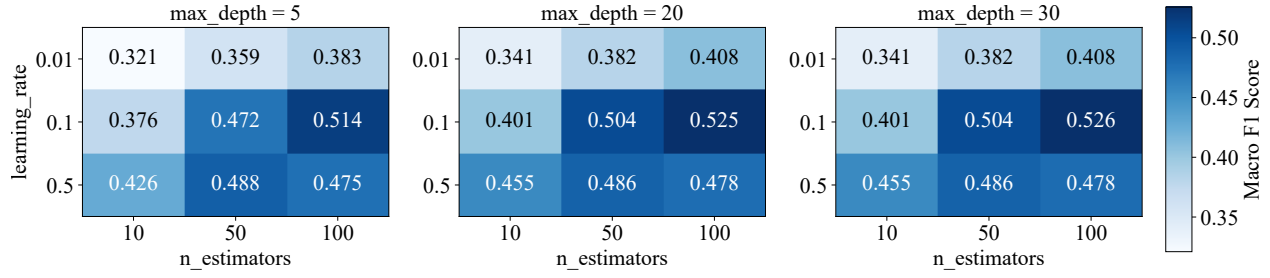
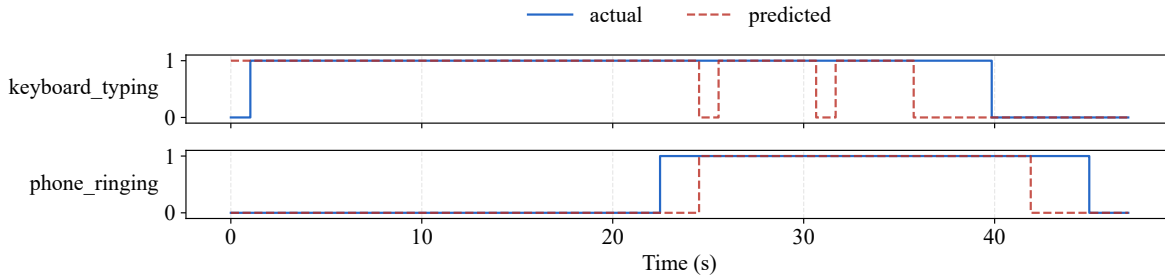


Figure 4: Hyperparameter Space of the GBM classifier. The darkest hyper parameter vector is chosen as the best model

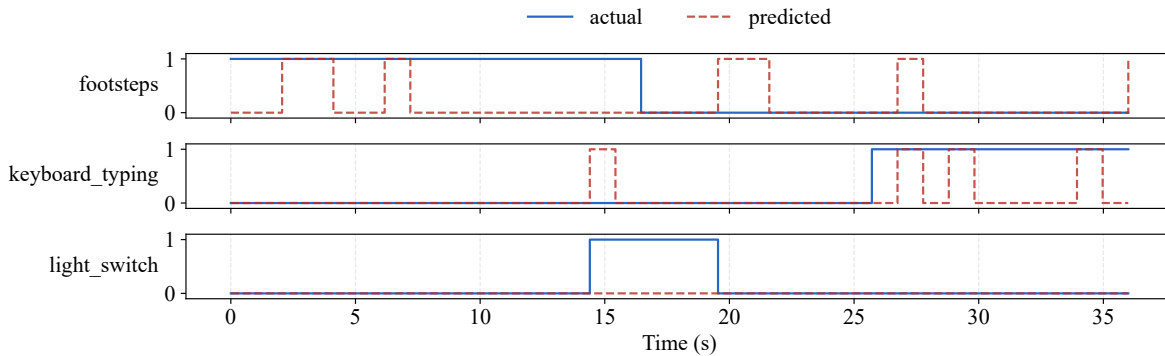
Table 5: Scores for the gradient boosting machine classifier predicting the unbiased test set			
	Model	Baseline	Improvement Factor
Macro Avg. Bal. Accuracy	0.8391	0.5004	1.68
Macro F1 Score	0.5113	0.0653	7.91
PR AUC	0.3298	0.0668	4.95

4 Case Study and Reflection

Two Test files were picked out and predicted with the *random forest classifier*, which can be seen in Figure 5. How the classifier performed can be seen in the respective figure description.



(a) Here, the classifier performed a good prediction. Description: I was typing on my keyboard, then my phone alarm rang (File ID: 001027)



(b) Here the classifier confused a light-switch with a keyboard. The light switch is a very rare class in comparison to the keyboard typing and also sounds very similar. The footsteps are also very poorly predicted. Description: I walked the the light switch, turned it on, moved the chair near me, started writing on my keyboard and clicked the mouse. (File ID: 001582)

Figure 5: Two Testfiles

4.1 Reflection

For comparison, a selection of per class confusion matrices can be seen for each classifier in Figure 6. From the low number overall positive predictions by the random forest classifier, one can see, that it predicts way more conservative as the GBM classifier. GBM predicts more positive events, consequently leading to a stronger false positive rate for e.g. footsteps. This could be because of class imbalance, as it is the most frequent class (compare Figure 2). The other dominating class *running_water* however, seems to be predicted more reliably.

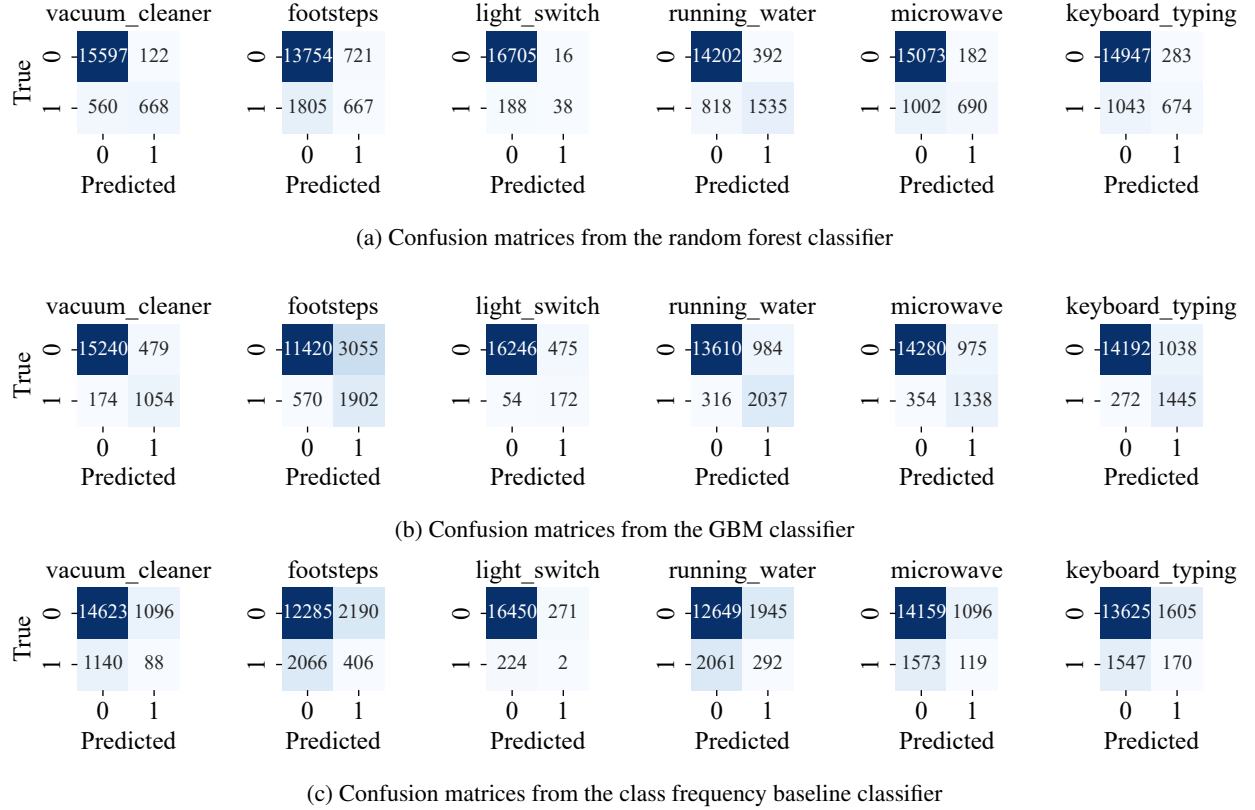


Figure 6: Selection of per class confusion matrices for each classifier

Both the random forest as well as the GBM classifier show improvement to the baseline confusion rates, as seen in Figure 6c.

5 Disclosure of LLM and AI Tool Use

In many cases, after the manual implementation of for example a classifier was done, LLM-Tooling often suggested an easier way of how it would be implemented using ready made sklearn features. This reduced the error susceptibility because less lines of code were needed, and also sped up the experiments, because many features support multi threading during training (for example the grid search).